



Impact Analysis - CodeSmile

Autore	Matricola
Daniele Pio Scaparra	NF22500101
Grazia Bonagura	NF22500105
Antonio Nappi	NF22500200

Indice

1. Introduzione.....	2
2. Change Request considerate.....	2
3. Tecnica di Impact Analysis adottata.....	2
4. Modello di Traceability.....	3
5. Modello di dipendenze (Call Graph concettuale).....	4
6. Change Request CR1 — Miglioramento del Detector In-Place APIs Misused.....	4
6.1 Descrizione della CR.....	4
6.2 Tecnica di Impact Analysis per CR1.....	4
6.3 Traceability Matrix (CR1).....	5
6.4 Impact Sets (CR1).....	5
6.5 Metriche (CR1).....	6
7. Change Request CR2 — Creazione del Call Graph del Progetto.....	8
7.1 Descrizione della CR.....	8
7.2 Tecnica di Impact Analysis per CR2.....	8
7.3 Traceability Matrix (CR2).....	8
7.4 Call Graph concettuale per l'Impact Analysis (CR2).....	9
7.5 Impact Sets (CR2).....	11
7.6 Metriche (CR2).....	13
8. Change Request CR3 — Analisi e Visualizzazione del Call Graph.....	15
8.1 Descrizione della CR3 — Analisi e Visualizzazione del Call Graph.....	15
8.2 Tecnica di Impact Analysis per CR3.....	15
8.3 Traceability Matrix (CR3).....	15

8.4 Call Graph concettuale per l'Impact Analysis (CR3).....	16
8.5 Impact Sets (CR3).....	17
8.6 Metriche (CR3).....	19

1. Introduzione

Il presente documento di Impact Analysis ha l'obiettivo di analizzare e valutare l'impatto delle Change Request approvate per il sistema **CodeSmile**, al fine di supportare le attività di modifica, riducendo il rischio di effetti collaterali e regressioni indesiderate.

L'Impact Analysis è condotta **prima dell'implementazione delle Change Request**, con lo scopo di:

- identificare le componenti inizialmente coinvolte dalle modifiche;
- stimare gli impatti diretti e indiretti sul sistema;
- fornire una base razionale per la pianificazione delle attività di sviluppo e testing;
- consentire un confronto strutturato tra impatti previsti e impatti effettivi (post-modifica).

2. Change Request considerate

Le Change Request oggetto della presente analisi sono:

- **CR1** — Miglioramento del detector *In-Place APIs Misused*
- **CR2** — Creazione del Call Graph del progetto
- **CR3** — Analisi e visualizzazione del Call Graph

Ciascuna Change Request presenta caratteristiche differenti in termini di natura della modifica (locale vs architetturale) e di propagazione potenziale degli impatti.

Per questo motivo, **la tecnica di Impact Analysis adottata varia in funzione della CR**, come descritto nelle sezioni successive.

3. Tecnica di Impact Analysis adottata

L'Impact Analysis è condotta adottando un **approccio ibrido**, che combina:

- **Traceability-based Impact Analysis**, per individuare le componenti direttamente coinvolte a partire dai concetti introdotti dalle Change Request;

- **Dependency-based Impact Analysis**, per stimare gli impatti indiretti attraverso l'analisi delle dipendenze tra componenti;
- **Call Graph reasoning**, utilizzato come modello concettuale delle relazioni di chiamata, limitatamente alle Change Request di natura architetturale.

Il livello di astrazione del modello di dipendenze (call graph) è selezionato in modo coerente con ciascuna Change Request, evitando analisi a grana eccessivamente fine quando non necessarie.

4. Modello di Traceability

Per supportare l'Impact Analysis è stato definito un modello di **traceability verticale** come illustrato nella figura sottostante.

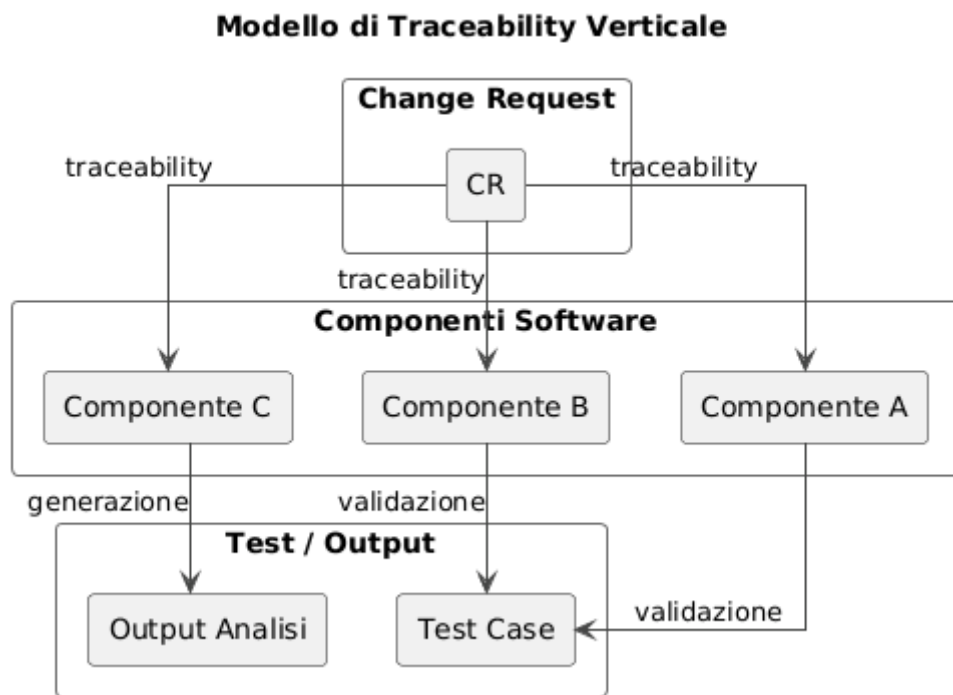


Fig.1 –Modello di Traceability Verticale che consente di mappare $CR \rightarrow Componenti\ Software \rightarrow Test$.

La traceability è costruita manualmente mediante:

- analisi della documentazione architetturale del sistema;
- ispezione statica della struttura dei package e dei moduli;
- analisi delle responsabilità dei componenti coinvolti.

La traceability matrix costituisce il riferimento principale per l'identificazione del **Starting Impact Set (SIS)**.

5. Modello di dipendenze (Call Graph concettuale)

Per le Change Request di natura architetturale (CR2 e CR3), l'Impact Analysis è supportata da un **call graph concettuale**, utilizzato esclusivamente a fini di previsione dell'impatto.

Caratteristiche del call graph utilizzato nell'Impact Analysis:

- rappresenta dipendenze tra **componenti, moduli o servizi**, non tra singole funzioni;
 - è parziale e focalizzato esclusivamente sulle componenti rilevanti per la Change Request;
 - è costruito manualmente mediante analisi statica del codice e delle responsabilità dei moduli
-

6. Change Request CR1 — Miglioramento del Detector In-Place APIs Misused

6.1 Descrizione della CR

La Change Request **CR1** ha l'obiettivo di migliorare il detector di code smell *In-Place APIs Misused*, intervenendo sulla logica semantica di rilevazione delle operazioni in-place il cui risultato non viene utilizzato correttamente.

La modifica mira ad ampliare la copertura delle API supportate (in particolare Pandas e NumPy), a ridurre la presenza di falsi positivi e falsi negativi e a rendere il comportamento del detector più aderente alla semantica delle librerie di data science analizzate.

La CR1 introduce una modifica locale e confinata alla regola di detection, senza alterare il flusso di esecuzione o le interfacce tra componenti del sistema

6.2 Tecnica di Impact Analysis per CR1

La CR1 introduce una modifica **locale e semantica**, confinata alla logica interna di un detector di code smell.

Pertanto, l'Impact Analysis per questa CR è condotta **esclusivamente tramite traceability-based impact analysis**, senza costruzione di un call graph.

La scelta di non utilizzare un call graph è motivata dal fatto che:

- la CR non introduce nuove relazioni di chiamata;

- non modifica il flusso di esecuzione del sistema;
- non altera le interfacce tra componenti.

6.3 Traceability Matrix (CR1)

CR1 – Traceability Matrix (Concetti → Componenti)

Concetto CR1	in_place_apis_misused.py	rule_checker.py	inspector.py
Implementazione detector IPA	✓		
Logica semantica in-place	✓		
Applicazione regole di detection		✓	
Analisi AST			✓
Riduzione falsi positivi / negativi	✓		

Fig.2 – CR1: Matrice di tracciabilità tra concetti funzionali e componenti software.

6.4 Impact Sets (CR1)

Lo **Starting Impact Set (SIS)** per la Change Request CR1 include le componenti **direttamente coinvolte dall'introduzione** e dal miglioramento della logica di detection dello smell In-Place APIs Misused.

L'identificazione del SIS è stata effettuata mediante **traceability-based impact analysis**, mappando i concetti funzionali introdotti dalla CR sui componenti software che ne implementano o supportano l'esecuzione.

Le componenti identificate sono:

- **detection_rules/generic/in_place_apis_misused.py**
Componente che implementa direttamente la regola di detection oggetto della CR1.
La modifica interviene sulla logica semantica del detector, ampliando la copertura delle API supportate e migliorando l'accuratezza della rilevazione.
- **components/rule_checker.py**
Modulo responsabile dell'applicazione delle regole di detection durante l'analisi statica.
Il comportamento osservabile di questo componente dipende direttamente dalla logica della regola migliorata.
- **components/inspector.py**
Componente che fornisce le informazioni strutturali (AST) utilizzate dal detector per analizzare l'uso delle API in-place.

È incluso nel SIS in quanto parte integrante del flusso di esecuzione della regola.

SIS_{CR1} :

`{detection_rules/generic/in_place_apis_misused.py, components/rule_checker.py, components/inspector.py}`

Il **Candidate Impact Set (CIS)** rappresenta l'insieme delle componenti che potrebbero essere impattate **indirettamente** dalla Change Request.

Nel caso della CR1, non sono stati individuati impatti indiretti su ulteriori componenti del sistema, in quanto:

- la modifica è confinata alla logica interna del detector;
- non vengono introdotte nuove dipendenze;
- non viene modificato il flusso di invocazione tra componenti.

Pertanto: $SIS_{CR1} = CIS_{CR1}$

L'**Actual Impact Set (AIS)** identifica le componenti **effettivamente modificate** durante l'implementazione della Change Request.

A seguito dell'implementazione della CR1, è stato modificato esclusivamente il modulo che implementa la regola di detection, senza interventi sui componenti di orchestrazione o di supporto.

$AIS_{CR1} = \{detection_rules/generic/in_place_apis_misused.py\}$

Il **False Positive Impact Set (FPIS)** include le componenti inizialmente considerate nel SIS/CIS che **non hanno richiesto modifiche** durante l'implementazione.

Nel caso della CR1, i seguenti componenti non sono stati modificati:

$FPIS_{CR1} = \{components/rule_checker.py, components/inspector.py\}$

Il **Discovered Impact Set (DIS)** rappresenta le componenti non previste inizialmente che sono emerse come impattate durante l'implementazione.

Durante l'implementazione della CR1 **non sono emersi componenti aggiuntivi** rispetto a quelli previsti nell'analisi iniziale.

$DIS_{CR1} = \emptyset$

6.5 Metriche (CR1)

Al fine di valutare la qualità dell'Impact Analysis condotta per la Change Request **CR1 – Miglioramento del detector In-Place APIs Misused**, sono state calcolate le metriche di **Precision** e

Recall dell'Impact Analysis, in accordo con la letteratura sull'analisi di impatto del software.

Le metriche sono definite come segue:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} \quad \Bigg| \quad Recall = \frac{|CIS \cap AIS|}{|AIS|}$$

dove:

- **CIS (Candidate Impact Set)** rappresenta l'insieme delle componenti potenzialmente impattate;
- **AIS (Actual Impact Set)** rappresenta l'insieme delle componenti effettivamente modificate.

Valori osservati

Per la CR1 risultano:

- $CIS_{CR1} = \{\text{detection_rules/generic/in_place_apis_misused.py, components/rule_checker.py, components/inspector.py}\}$
- $AIS_{CR1} = \{\text{detection_rules/generic/in_place_apis_misused.py}\}$

Ne consegue che:

$$Precision = \frac{1}{3} \approx 0.33 \quad \Bigg| \quad Recall = \frac{1}{1} = 1$$

Interpretazione dei risultati

Il valore di **Recall pari a 1.00** indica che l'Impact Analysis ha correttamente identificato tutte le componenti che sono state effettivamente modificate durante l'implementazione della CR1, senza omissioni.

Il valore di **Precision pari a 0.33** riflette invece un approccio intenzionalmente **conservativo** nella

definizione del CIS, che include anche componenti di supporto (es. `rule_checker.py`, `inspector.py`) ritenute potenzialmente coinvolte nella fase iniziale dell'analisi, ma che non hanno richiesto modifiche effettive.

Tale risultato è coerente con l'obiettivo dell'Impact Analysis, che privilegia la **completezza** (assenza di falsi negativi) rispetto alla massimizzazione della precisione nelle fasi preliminari di pianificazione delle modifiche.

7. Change Request CR2 — Creazione del Call Graph del Progetto

7.1 Descrizione della CR

La Change Request **CR2** introduce una nuova funzionalità architetturale che consente la **generazione automatica del call graph** del progetto analizzato da CodeSmile.

La modifica estende il flusso di analisi esistente, permettendo di estrarre e rappresentare esplicitamente le relazioni di chiamata tra funzioni e metodi del codice.

Il call graph generato costituisce una base strutturale per una migliore comprensione dell'architettura del progetto e per le successive attività di analisi avanzata e visualizzazione previste dalla CR3.

7.2 Tecnica di Impact Analysis per CR2

La Change Request **CR2** introduce una **nuova funzionalità architetturale** che estende il flusso di analisi del sistema CodeSmile, consentendo la generazione del **call graph del progetto analizzato**.

A differenza della CR1, la CR2:

- introduce **nuove responsabilità funzionali**;
- estende il flusso di esecuzione esistente;
- può avere **impatti strutturali** su più componenti.

Pertanto, l'Impact Analysis per la CR2 è condotta mediante un approccio combinato:

- **traceability-based impact analysis**, per individuare le componenti direttamente coinvolte a partire dai concetti introdotti dalla CR;
- **dependency-based impact analysis**, supportata da un **call graph concettuale a livello di moduli/classi**, utilizzato esclusivamente a fini di previsione dell'impatto.

7.3 Traceability Matrix (CR2)

CR2 – Traceability Matrix (Concetti → Componenti)

Concetto CR2	cli_runner.py	project_analyzer.py	inspector.py
Estensione CLI (nuovo flag)	✓		
Orchestrazione analisi		✓	
Generazione Call Graph		✓	✓
Analisi strutturale del codice			✓
Riutilizzo informazioni AST			✓

Fig. 3 – CR2: Matrice di tracciabilità tra concetti introdotti dalla Change Request e componenti software.

7.4 Call Graph concettuale per l’Impact Analysis (CR2)

CR2 - Call Graph Concettuale per Impact Analysis

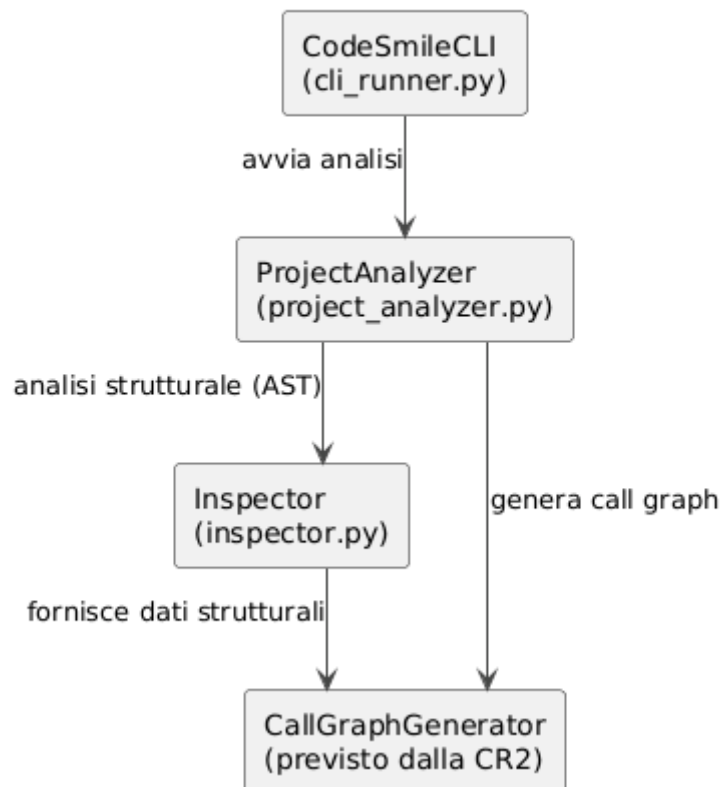


Fig. 4 – CR2: Call graph concettuale utilizzato per l’Impact Analysis.

Scopo del call graph

Il call graph costruito per la Change Request **CR2** ha lo scopo di **supportare la dependency-based impact analysis**, consentendo di:

- identificare le relazioni di chiamata rilevanti tra i componenti coinvolti;
- stimare la possibile propagazione degli impatti;
- derivare il **Candidate Impact Set (CIS)** a partire dal SIS.

Il call graph è costruito **prima dell'implementazione**, a livello **concettuale**, e **non rappresenta il call graph completo del codice**.

Livello di astrazione adottato

Per la CR2 è stato adottato un livello di astrazione **a moduli/classi principali**, coerente con la natura architetturale della modifica.

In particolare:

- sono inclusi solo i componenti direttamente rilevanti per la generazione del call graph;
- non sono rappresentate le singole funzioni o i dettagli interni delle regole di detection;
- le relazioni indicano **invocazioni o dipendenze operative**, non semplici import.

Componenti inclusi nel call graph

I nodi del call graph concettuale sono:

- CodeSmileCLI (CLI)
- ProjectAnalyzer
- Inspector
- CallGraphGenerator (*componente previsto dalla CR, non ancora implementato*)

Relazioni di chiamata considerate

Le relazioni rilevanti per la CR2 sono:

1. La **CLI** invoca il ProjectAnalyzer per avviare l'analisi del progetto.
2. Il ProjectAnalyzer coordina l'analisi e richiama:
 - l'Inspector per ottenere le informazioni strutturali del codice;
 - il CallGraphGenerator per costruire il call graph.
3. L'Inspector fornisce i dati necessari alla costruzione del call graph, ma non ne controlla la generazione.

7.5 Impact Sets (CR2)

SIS (Starting Impact Set)

Lo **Starting Impact Set (SIS)** per la CR2 include le componenti **direttamente coinvolte** dall'introduzione della funzionalità di generazione del call graph.

L'identificazione del SIS è stata effettuata mediante **traceability-based impact analysis**, mappando i concetti introdotti dalla CR sui componenti che:

- orchestrano l'analisi del progetto;
- costruiscono o forniscono le informazioni strutturali del codice;
- espongono la funzionalità all'utente tramite la CLI.

Concetti chiave della CR2

La CR2 introduce i seguenti concetti funzionali:

- generazione del call graph;
- estensione della CLI con una nuova opzione/flag;
- orchestrazione della nuova pipeline di analisi;
- riutilizzo delle informazioni AST già disponibili.

Componenti direttamente coinvolte

Sulla base della traceability concetto → componente, sono state identificate le seguenti componenti:

- **cli/cli_runner.py**
Responsabile dell'interfaccia a linea di comando.
La CR2 richiede l'estensione della CLI per consentire all'utente di richiedere la generazione del call graph.
- **components/project_analyzer.py**
Componente che orchestra l'analisi del progetto.
È direttamente coinvolto in quanto deve coordinare l'esecuzione della nuova funzionalità di generazione del call graph all'interno del flusso di analisi esistente.
- **components/inspector.py**
Componente responsabile della costruzione dell'AST e dell'analisi strutturale del codice.
Le informazioni fornite dall'Inspector costituiscono la base per la costruzione del call grap

SIS_{CR2}

```
{cli/cli_runner.py, components/project_analyzer.py, components/inspector.py  
}
```

Nota metodologica

Il SIS include esclusivamente le componenti che:

- implementano direttamente i concetti introdotti dalla CR2;
- partecipano in modo essenziale all'esecuzione della nuova funzionalità.

Componenti di presentazione dei risultati (es. report generator) o di detection degli smell non sono incluse nel SIS, in quanto non direttamente coinvolte nella generazione del call graph.

CIS (Candidate Impact Set)

Il **Candidate Impact Set (CIS)** rappresenta l'insieme delle componenti che **potrebbero essere impattate indirettamente** dalla CR2, sulla base delle dipendenze individuate tramite il call graph concettuale.

A partire dal call graph definito nella sezione precedente, si osserva che:

- il ProjectAnalyzer funge da nodo di orchestrazione centrale;
- l'introduzione della generazione del call graph può influenzare i moduli che:
 - ricevono i risultati dell'analisi;
 - estendono il flusso di elaborazione esistente;
 - gestiscono l'output dell'analisi.

Sulla base di tali considerazioni, il CIS include:

- tutte le componenti del SIS;
- eventuali componenti responsabili della gestione dei risultati dell'analisi.

Tuttavia, nella fase di previsione dell'impatto, non sono stati individuati ulteriori componenti chiaramente dipendenti in modo diretto dalla nuova funzionalità.

Pertanto: $SIS_{CR2} = CIS_{CR2}$

AIS (Actual Impact Set)

L'**Actual Impact Set (AIS)** identifica le componenti **effettivamente modificate** durante l'implementazione della CR2.

A seguito dell'implementazione, le modifiche hanno riguardato:

- l'estensione della CLI per supportare la generazione del call graph;
- l'orchestrazione della nuova funzionalità nel flusso di analisi;
- l'introduzione di un nuovo modulo dedicato alla costruzione del call graph.

AIS_{CR2}

`{cli/cli_runner.py, components/project_analyzer.py, components/call_graph_generator.py}`

FPIS (False Positive Impact Set)

Il **False Positive Impact Set (FPIS)** include le componenti che erano state inizialmente considerate nel SIS/CIS ma che **non hanno richiesto modifiche** durante l'implementazione.

Nel caso della CR2:

- l'Inspector era stato incluso nel SIS in quanto fornitore delle informazioni strutturali;
- tuttavia, la sua logica non ha richiesto modifiche, poiché la generazione del call graph è stata isolata in un nuovo componente dedicato.

$FPIS_{CR2} = \{components/inspector.py\}$

DIS (Discovered Impact Set)

Il **Discovered Impact Set (DIS)** rappresenta le componenti **non previste inizialmente** che sono emerse come necessarie durante l'implementazione.

Nel caso della CR2, è emerso un nuovo componente:

- **components/call_graph_generator.py**
Introdotta per separare la logica di generazione del call graph dall'analisi degli smell, migliorando modularità e manutenibilità.

$DIS_{CR2} = \{components/call_graph_generator.py\}$

7.6 Metriche (CR2)

Metriche di Impact Analysis

Al fine di valutare la qualità dell'Impact Analysis condotta per la Change Request **CR2 – Creazione del Call Graph del Progetto**, sono state calcolate le metriche di **Precision** e **Recall** dell'Impact Analysis,

Valori osservati

Per la CR2 risultano:

- CIS_{CR2}
`{cli/cli_runner.py, components/project_analyzer.py, components/inspector}`
- AIS_{CR2}
`{cli/cli_runner.py, components/project_analyzer.py, components/call_graph_generator.py}`

L'intersezione tra CIS e AIS è quindi:

$$CIS_{CR2} \cap AIS_{CR2} = \{\text{cli_runner.py, project_analyzer.py}\}$$

Da cui si ottengono i seguenti valori:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{2}{3} \approx 0.67$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{2}{3} \approx 0.67$$

Interpretazione dei risultati

I valori di precision e recall evidenziano che l'Impact Analysis per la CR2 ha fornito una previsione **complessivamente corretta**, individuando i principali componenti coinvolti dalla modifica.

La presenza di:

- un **falso positivo** (inspector.py, incluso nel CIS ma non modificato),
- un **componente scoperto** (call_graph_generator.py, emerso in fase di implementazione),

è coerente con la natura **architetturale ed evolutiva** della Change Request, che ha richiesto una rifattorizzazione della logica inizialmente prevista attraverso l'introduzione di un nuovo modulo dedicato.

8. Change Request CR3 — Analisi e Visualizzazione del Call Graph

8.1 Descrizione della CR3 — Analisi e Visualizzazione del Call Graph

La Change Request **CR3** estende le funzionalità introdotte dalla CR2, introducendo strumenti dedicati all'**analisi e alla visualizzazione del call graph** generato.

La modifica consente di elaborare il call graph per individuare strutture rilevanti (es. cicli e nodi critici), calcolare metriche di dipendenza e produrre rappresentazioni grafiche esportabili del grafo.

La CR3 migliora l'interpretabilità dei risultati forniti dal sistema, supportando le attività di comprensione architetturale e manutenzione del codice analizzato.

8.2 Tecnica di Impact Analysis per CR3

La Change Request **CR3** introduce funzionalità di **analisi e visualizzazione del call graph** generato dal sistema, estendendo ulteriormente le capacità introdotte dalla CR2.

La CR3:

- non modifica la logica di generazione del call graph;
- introduce nuove modalità di **elaborazione, analisi e presentazione** dei dati;
- coinvolge più componenti a valle del flusso di analisi.

Pertanto, l'Impact Analysis per la CR3 è condotta mediante un approccio combinato:

- **traceability-based impact analysis**, per individuare le componenti direttamente coinvolte a partire dai concetti introdotti dalla CR;
- **dependency-based impact analysis**, supportata da un **call graph concettuale a livello di servizi e componenti**, utilizzato per stimare la propagazione degli impatti.

8.3 Traceability Matrix (CR3)

Obiettivo della tracciabilità

La matrice di tracciabilità per la CR3 è utilizzata per:

- mappare i concetti introdotti dalla CR sui componenti software coinvolti;
- giustificare formalmente lo **Starting Impact Set (SIS)**;
- fornire una base per l'analisi delle dipendenze a valle del call graph generato.

CR3 – Traceability Matrix (Concetti → Componenti)

Concetto CR3	call_graph_generator.py	call_graph_analyzer.py	report_generator.py	cli_runner.py
Analisi del call graph		✓		
Calcolo metriche (centralità, cicli)		✓		
Visualizzazione del call graph			✓	
Esportazione risultati (PNG/SVG)			✓	
Estensione CLI				✓
Riutilizzo call graph generato	✓	✓		

Fig.5 – CR3: Matrice di tracciabilità tra i concetti introdotti dalla Change Request e i componenti software coinvolti.

8.4 Call Graph concettuale per l’Impact Analysis (CR3)

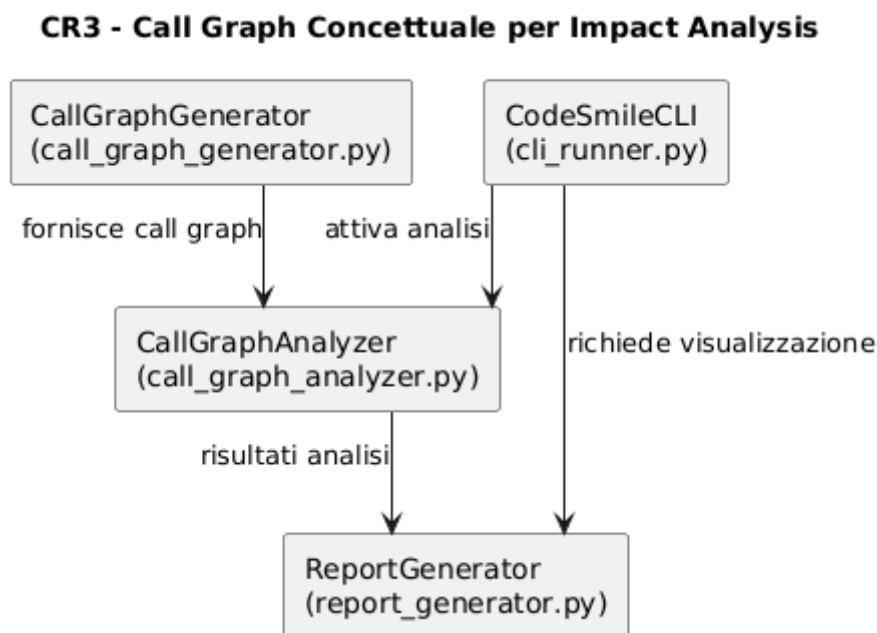


Fig 6. – CR3: Call graph concettuale utilizzato per la dependency-based impact analysis.

8.4.1 Scopo del call graph

Il call graph costruito per la Change Request **CR3** ha lo scopo di supportare la **dependency-based impact analysis**, consentendo di:

- identificare le dipendenze tra i componenti che analizzano e visualizzano il call graph;
- stimare la propagazione degli impatti a valle della funzionalità introdotta dalla CR2;
- derivare il **Candidate Impact Set (CIS)** a partire dallo **Starting Impact Set (SIS)**.

Il call graph è costruito **prima dell'implementazione**, a livello **concettuale**, e non rappresenta il call graph operativo prodotto dal sistema.

8.4.2 Livello di astrazione adottato

Per la CR3 è stato adottato un livello di astrazione **a componenti e servizi principali**, coerente con la natura funzionale e trasversale della modifica.

In particolare:

- sono inclusi solo i componenti che elaborano, analizzano o presentano il call graph;
- non sono rappresentati dettagli implementativi o strutture interne;
- le relazioni indicano **flussi di dipendenza funzionale**, non chiamate di basso livello.

8.4.3 Componenti inclusi nel call graph

I nodi del call graph concettuale per la CR3 sono:

- **CallGraphGenerator** (introdotto in CR2)
- **CallGraphAnalyzer** (nuovo componente previsto dalla CR3)
- **ReportGenerator**
- **CodeSmileCLI**

8.4.4 Relazioni di dipendenza considerate

Le relazioni rilevanti per la CR3 sono:

- il CallGraphGenerator produce il call graph di base;
- il CallGraphAnalyzer utilizza il call graph per calcolare metriche e individuare strutture rilevanti (es. cicli);
- il ReportGenerator utilizza i risultati dell'analisi per produrre rappresentazioni visuali ed esportabili;
- la **CLI** consente all'utente di richiedere l'analisi e la visualizzazione del call graph.

8.5 Impact Sets (CR3)

SIS (Starting Impact Set)

Lo **Starting Impact Set (SIS)** per la CR3 include le componenti **direttamente coinvolte** dall'introduzione delle funzionalità di analisi e visualizzazione del call graph.

Sulla base della traceability e del call graph concettuale, il SIS è composto da:

```
{components/call_graph_generator.py, components/call_graph_analyzer.py, components/report_generator.py, cli/cli_runner.py}
```

Nota metodologica

Il SIS include anche componenti introdotte in precedenti Change Request (es. `call_graph_generator.py`), in quanto la CR3 **riutilizza e estende funzionalità esistenti**, generando potenziali impatti a valle del flusso di analisi.

CIS (Candidate Impact Set)

Il **Candidate Impact Set (CIS)** rappresenta l'insieme delle componenti che **potrebbero essere impattate indirettamente** dalla CR3, sulla base delle dipendenze individuate tramite la traceability e il call graph concettuale.

A partire dal call graph concettuale per la CR3, si osserva che:

- la CR3 riutilizza il call graph generato dalla CR2;
- introduce nuove fasi di analisi e visualizzazione a valle del flusso di generazione;
- coinvolge componenti che elaborano, presentano o espongono i risultati all'utente.

Sulla base di tali considerazioni, il CIS include:

- tutte le componenti del **SIS**;
- i componenti che forniscono il call graph come input all'analisi e alla visualizzazione.

Pertanto, il **CIS per la CR3** è definito come:

CIS_{CR3}

```
{components/call_graph_generator.py, components/call_graph_analyzer.py, components/report_generator.py, cli/cli_runner.py}
```

AIS (Actual Impact Set)

L'**Actual Impact Set (AIS)** identifica le componenti **effettivamente modificate o introdotte** durante l'implementazione della CR3.

A seguito dell'implementazione della CR3, le modifiche hanno riguardato:

- l'introduzione di un nuovo componente dedicato all'analisi del call graph;
- l'estensione del modulo di report generation per supportare la visualizzazione del grafo;
- l'estensione della CLI per consentire l'attivazione delle nuove funzionalità.

Il **AIS per la CR3** risulta quindi:

AIS_{CR3}

```
{components/call_graph_analyzer.py, components/report_generator.py, cli/cli_runner.py}
```

FPIS (False Positive Impact Set)

Il **False Positive Impact Set (FPIS)** include le componenti che erano state inizialmente considerate nel SIS/CIS ma che **non hanno richiesto modifiche** durante l'implementazione della CR3.

Nel caso della CR3:

- il `call_graph_generator.py` è stato incluso nel SIS/CIS in quanto fornitore del call graph;
- tuttavia, la sua logica non ha richiesto modifiche, poiché la CR3 riutilizza il call graph così come prodotto dalla CR2.

Pertanto:

$FPIS_{CR3} = \{\text{components/call_graph_generator.py}\}$

DIS (Discovered Impact Set)

Il **Discovered Impact Set (DIS)** rappresenta le componenti **non previste inizialmente** che sono emerse come necessarie durante l'implementazione.

Nel caso della CR3, **non sono emerse componenti aggiuntive** rispetto a quelle previste nell'Impact Analysis iniziale.

$DIS_{CR3} = \emptyset$

8.6 Metriche (CR3)

Metriche di Impact Analysis

Al fine di valutare la qualità dell'Impact Analysis condotta per la Change Request **CR3 – Analisi e Visualizzazione del Call Graph**, sono state calcolate le metriche di **Precision** e **Recall** dell'Impact Analysis

Valori osservati

Per la CR3 risultano:

- CIS_{CR3}

```
{components/call_graph_generator.py, components/call_graph_analyzer.py, components/report_generator.py, cli/cli_runner.py}
```

- AIS_{CR3}

```
{components/call_graph_analyzer.py, components/report_generator.py, cli/cli_runner.py}
```

L'intersezione tra CIS e AIS è quindi:

$CIS_{CR3} \cap AIS_{CR3} = \{\text{call_graph_analyzer.py, report_generator.py, cli_runner.py}\}$

Da cui si ottengono i seguenti valori:

$$Precision = \frac{|CIS \cap AIS|}{|CIS|} = \frac{3}{4} = 0.75$$

$$Recall = \frac{|CIS \cap AIS|}{|AIS|} = \frac{3}{3} = 1$$

Interpretazione dei risultati

Il valore di **Recall pari a 1.00** indica che l'Impact Analysis per la CR3 ha correttamente identificato tutte le componenti che sono state effettivamente modificate durante l'implementazione, senza omissioni.

Il valore di **Precision pari a 0.75** riflette la presenza di un singolo **falso positivo** (`call_graph_generator.py`), incluso nel CIS in quanto componente a monte del flusso di analisi, ma non modificato durante l'implementazione della CR3.

Nel complesso, i risultati evidenziano una buona accuratezza dell'analisi predittiva per una Change Request di tipo **evolutivo**, caratterizzata da una propagazione degli impatti prevalentemente a valle del flusso di analisi.

Nota metodologica

Come per le altre Change Request, le metriche di Impact Analysis valutano esclusivamente la **qualità della previsione dell'impatto** e non l'efficacia funzionale delle funzionalità introdotte, la cui validazione è demandata alle attività di testing e al documento post-modification.